

Student Models in Computer-aided Instruction

JOHN A. SELF

Centre for Computer Studies, University of Leeds, Leeds, U.K.

(Received 15 November 1973)

In order to individualize instruction, a computer-aided instruction (CAI) program must maintain a "student model", i.e. a representation of the hypothesized knowledge state of the student. We propose a student model consisting of a set of programs to represent the student's knowledge state. Teaching proceeds after a comparative evaluation of student and teacher programs, and learning is represented by direct modification of the student model. The advantages of an explicit procedural model are illustrated by considering a program which maintains such a model. Theoretical and practical difficulties facing such an approach are also discussed.

Introduction

The *raison d'être* of computer-aided instruction (CAI) is often said to be its ability to provide individualized teaching, i.e. to evaluate a teaching function one of whose arguments is a representation of the student. The knowledge determining a teaching program's overall performance can be classified under three headings:

- (a) knowledge of how to teach (which includes knowledge of students in general);
- (b) knowledge of what is being taught;
- (c) knowledge of who is being taught (i.e. knowledge of one student in particular).

Of these, only type (c) knowledge is specifically relevant to the aim of "individualization". This knowledge will be called the "student model".

Since the term "student model" has been used rather freely in the CAI literature, it is to be emphasized that the "model" should serve its customary purpose; to an observer O , an object M_s is a model of an object S to the extent that O can use M_s to answer questions about S . If we identify S as our student and let O pose the question "What is the current knowledge state of S ?", then we have a means of evaluating an M_s (a student model); that is to what extent does the M_s enable a human teacher or the teaching program itself (which are two possible O s) to determine how much S knows at any time?

The aims of this article are (1) to argue that CAI programs could develop a more natural representation of the teaching/learning process by maintaining a student model composed of a set of programs representing the student's hypothesized knowledge state, and (2) to illustrate the technique, and to clarify advantages and disadvantages of the approach, by considering an implementation of such a teaching program.

A Survey of Student Models

In considering computational formalizations of student models, it is necessary to distinguish carefully between type (b) and type (c) knowledge (referring to the above classification). The former is independent of the latter (and may therefore be considered constant over all teaching sessions), but not necessarily *vice versa*. In fact, a subset of type (c) knowledge should indicate the type (b) knowledge assimilated. For this reason, it will be necessary to consider representations of type (b) knowledge later.

The M_s s of most existing CAI programs are similar to those provided by author language systems. In these, the student receives individualized instruction to the extent that the program may branch to different nodes in a prespecified teaching sequence, depending on the response matched and on the values of a set of changeable registers. Since it is only the latter which is carried from node to node, it is this, the set of registers, which constitutes the M_s . The implication is that a program path may be found, as a function of register contents, such that a threshold level of understanding is attained and that the registers enable such levels to be measured.

The most obvious deficiency of author language systems—the need for a laborious prespecification and refinement of display text and anticipated responses—leads naturally to generative systems, in which problems and comments are not prestored but generated dynamically. Clearly, this generation should be a function of an M_s . However, particular M_s s found in practical generative systems are not significant improvements on those found in author language systems. For example, Peplinski (1970) simply records the number of questions and the number of correct answers, and this record is used to determine when to generate more difficult questions. Thus, the M_s is a set of just two changeable registers. Similarly, in Taylor's system (Taylor, 1968), "the programmer can set any number of indices for the machine to keep track of" and the calling of scripts (but not the execution of them) can depend on some pre-determined profile of indices. And Siklossy's discussion of his program to teach set theory (Siklossy, 1971) contains no mention of type (c) knowledge.

Generative CAI programs can be divided into two classes: those in which the type (b) knowledge is implicit, and those in which it is explicit. Only in the latter case can type (c) knowledge refer directly to type (b) knowledge. Most generative CAI programs belong in the former category. For example, Peplinski embeds type (b) knowledge in the ALGOL 60 procedures which solve the problems generated. Siklosy expresses type (b) knowledge as LISP functions, which makes them available as data, under interpretation, although, with one exception mentioned later, he does not propose to make use of this.

In generative programs of the latter category, we have, in theory, the advantage that the M_s could represent the student's knowledge state by referring directly to the type (b) knowledge to indicate that which has been learnt. In practice, M_s s have been rather simpler.

In Carbonell's system (Carbonell, 1970), in which type (b) knowledge is expressed as a "semantic network" (about which there will be more later), there appears to be only one element of the total knowledge which belongs in type (c). This is his "temporary tags" which are inserted at strategic points in the semantic network to indicate (1) whether a particular node of the network has been asked, and, if so, whether answered correctly or not, and (2) how far into the network a system answer to a question of the form "tell me about . . ." has reached. Wexler (1970) also inserts tags into a semantic network to indicate whether an item has been asked or presented to the student. His system also maintains a set of registers, as in author language systems, which "may be used to decide between alternatives in a complex remedial strategy devised by a teacher", although, as is often the case, there are a host of arbitrary conventions and restrictions which make such an analysis difficult. Brown, Burton & Zdybel (1972) represent part of the type (b) knowledge as a semantic network, but make no use of "tags" as above. Instead, they maintain a student model (called an "external state vector") consisting of a set of flags (or registers) which are altered in accordance with the kind of material the student is currently exploring. These flags do not refer to type (b) knowledge, but are intended to indicate the presuppositions behind student questions. Each flag can take one of a prespecified small number of values.

To summarize then, most CAI systems maintain an M_s consisting of a set of registers, the contents of which are interpreted by type (a) knowledge, plus tags relating to explicit type (b) knowledge, if this exists. Since such programs have changeable registers and can transfer control depending on the state of these registers, they have the power of Turing machines and can therefore conduct any type of teaching dialogue which could be produced

by any machine. Any difficulty is a practical rather than theoretical one. Any advantages of an alternative M_s lie in the way in which it gives a natural and understandable representation of the teaching/learning process, and facilitates the implementation of effective teaching programs. CAI should aim not only to improve the efficiency of educational processes, but to work out models of the processes themselves.

A Procedural Student Model

The designer of a CAI program is concerned to define behavioural objectives, if only to provide a means of evaluation. The attainment of such objectives is determined by asking questions. Answering a question seems to involve evaluating a program, subroutines of which may be processes of information retrieval, calculation, hypothesis formation and evaluation, and problem subdivision. The natural way to represent one component of type (c) knowledge, therefore, is as a set of programs—the answers predicted by this set of programs should agree with those of the student. As the student learns, so his M_s should change (by modification of the set of programs), perhaps towards a set of corresponding programs expressing type (b) knowledge.

The need to relate type (c) knowledge more closely to type (b) knowledge is foreseen by both Carbonell and Wexler. Carbonell proposes that “a student’s input–output behaviour can be accounted for by introducing small perturbations in the semantic network”. Similarly, Wexler considers that erasures from the network “might be used in a dynamic model of a student’s presumed knowledge.” Neither proposal reached the top of the stack of priorities. However, two comments are in order: first, these changes are presumably to be made to a copy of the network rather than to the network itself (else the program will begin making the same errors as the student), and secondly, it is assumed that the organization of knowledge termed a “semantic network” can be usefully compared to that which is to be learned by the student.

Thus, our concern with type (c) knowledge involves a consideration of type (b) knowledge.

Knowledge Systems

In CAI, one seeks descriptive, rather than normative, knowledge representations, in order that the program’s problem-solving processes should be comparable with those of a student.

The apparent division of CAI type (b) knowledge into programs [Peplinski (1970), Siklossy (1971)] and data [semantic networks—Wexler (1970), Carbonell (1970), Brown *et al.* (1972)] is an illusion. To say that knowledge is organized as a network is to say only that items of knowledge are inter-related—the important questions are concerned with how that network is interpreted. That this interpretation is not obvious is evidenced by the fact that the three CAI systems built round semantic networks have different interpretation mechanisms. Carbonell's network is interpreted in the standard Quillian way (Quillian, 1969), using its superpart–superconcept organization to permit processing and deduction. In Wexler's case, the teacher is required to define specific derivational processes (with the intended advantage that these processes are available for purposes of expository and remedial instruction). Brown *et al.* (1972) compromise by providing some specialized routines to perform a limited class of inferences, which the user may expand by adjoining new inferencing procedures to the executive which processes the network.

As we are seeking descriptive representations, we must consider the experimental evidence which is invariably quoted (Collins and Quillian, 1969) as indicating that knowledge is in fact organized as a “semantic network”. These experiments confirmed that retrieval times were proportional to the number of levels separating nodes (although results indicated that the processes of retrieving a property at a node and moving up a level are parallel ones, unlike the computer realizations), but utterly failed to explain response times when the information presented is contradicted by or absent from the network. Since a student will often have incorrect or inadequate information, this is a result which can hardly be ignored. The question–answering process is considerably more complicated (Norman, 1972) than the simple information retrieval process encouraged by semantic networks. There are clearly many open questions concerning a human's information processing methods.

That not all type (b) knowledge can be conveniently encoded as a semantic network is recognized in the system of Brown *et al.* (1972) which, in order to answer questions concerning causal relationships between events, has part of its type (b) knowledge represented as a set of augmented finite-state automata. Question–answering is then accomplished by running the global automaton and recording all possible global state changes. This trace of the program's execution provides the means to answer questions concerning causal chains of events and transitions, and the reasons for them.

This progression from “network” to “program” representation of type (b) knowledge is paralleled by recent trends in the field of artificial intelligence.

Here, an alternative to network approaches was devised in which type (b) knowledge was represented as axioms in a formal logical system, usually first-order predicate logic. Since an equivalence can be established between semantic networks and predicate logic, the advantages of the latter lie essentially in the fact that expressions in predicate logic have a standard interpretation. Apart from a proposal by Coles (1969), methods derived from formal logic and theorem-proving have had no impact on CAI. The axiomatic representation of knowledge does not provide a descriptive model nor an adequate normative one, since uniform proof procedures are inherently inefficient. These criticisms have led to the "procedural embedding of knowledge" thesis (Hewitt, 1972), which holds that intellectual structures should be analysed through their procedural analogues. In practice (Sussman, Winograd & Charniak, 1972; Sussman & McDermott, 1972), this means that the declarative axioms of formal logic are written as imperative procedures in a general goal-oriented language interpreted by a multiprocessing control structure with respect to the current state of knowledge of the system. These procedures may well contain explicit strategic information to control searches. The difficulties experienced with earlier attempts to represent knowledge by programs (i.e. that interactions between programs soon become unmanageable) are mitigated by having procedures automatically invoked when certain events occur, rather than called explicitly. Such attempts to design programming languages which formalize directly the foundations of problem-solving, and which require solutions to be expressed in terms of actions and purposes, encourage the view that it is becoming possible to express a student model in terms of computational processes which (it is hypothesized) that the student has conceptualized.

An Implementation

This section will discuss the implementation of a teaching program which maintains a "student model" intended to provide a more transparent answer to the paradigm question of the introduction, i.e. "What is the current knowledge state of the student?"

In order to provide a context, we will consider the dialogue of Fig. 1.* In general terms, we see that the teacher is trying to teach the student not particular items of information, but the relations between these items and how to establish such relations.

* This dialogue is a slightly modified version of one originally discussed by R. C. Anderson, University of Illinois, while Fulbright-Hayes Fellow at the Computer Based Learning Project, University of Leeds, during a seminar entitled "The structure and function of a Socratic teacher".

1. Teacher: Which is likely to be colder, Newfoundland or Montana?
2. Student: Newfoundland.
3. Please give your reasons for answering Newfoundland.
4. Newfoundland is further north.
5. Yes, that's true. Are you arguing then, that if you take any two places in the Northern Hemisphere, the one which is further north will have the lower average temperature?
6. Yes, I guess so.
7. I'll tell you now that Montana has lower average temperatures than Newfoundland. Does this fact cause you to change your reasoning?
8. Yes.
9. In what way?
10. Being further north isn't as important as I thought.
11. Please try to be more precise. Would you, for instance, say that if you take any two places in the Northern Hemisphere, the one furthest south has the colder temperature?
12. No, I wouldn't say that.
13. Would you say that how far north a place is has no effect on temperature?
14. No, I know it has some effect.
15. Yes, you're right. Distance north does affect temperature. Yet, even though it is further north, Newfoundland has higher average temperatures than Montana. What must this mean?
16. There must be some other factor.
17. Yes! Right! What could this factor be?
18. I don't have any idea.
19. Why don't you look at your map of North America. Do you see any differences between Montana and Newfoundland?
20. Montana is in the centre of the country. Newfoundland is on the ocean.
21. Do you suppose that distance from the ocean affects temperature?
22. I'm not sure. It would just be a guess.
23. True! The name for such a guess is an hypothesis. Supposing the hypothesis were correct, what exactly would you predict?
24. The further a place is from the ocean, the lower the temperature will be.
25. How could you test your hypothesis?
26. By comparing temperatures of places different distances from the ocean.
27. Very good. Let's do that. Suppose we take St Louis, Missouri. Which would be best to compare, Atlanta, Georgia, or Washington DC . . . ?
28. I would pick Washington.
29. Why?
30. Because it's at the same latitude as St Louis.
31. Why is that important?
32. Well, if Atlanta were warmer, I wouldn't know whether it was because it was nearer the ocean or further south.

FIG. 1. An instructional dialogue.

It is instructive to consider the derivation of the opening question. In Wexler's system, questions are generated after finding items in the network which match a teacher-specified "skeleton" pattern and then presented in a random order. In Carbonell's system, the teacher specifies an "agenda", which in practice consists of one element (e.g. South America). From this, a sub-context (e.g. Venezuela) is generated, initially at random since there are no tags, and this process is repeated until the system reaches an "item of knowledge", which is then asked. However, in Fig. 1, the subsequent

```

1. (PROG (MT MS X Y Z))
2. (LET F1 FACT (FURTHERINLAND MONTANA NEWFOUNDLAND))
3. (LET F2 FACT (FURTHERNORTH NEWFOUNDLAND MONTANA))
4. (LET T1 INFER (X Y) (COLDER ?X ?Y)
5.   (FACT (FURTHERNORTH ?X ?Y))
   (FACT (FURTHERINLAND ?X ?Y)))
6. (LET T2 INFER (X Y Z M1 M2 A B)
7.   (CONTRADICTION ?X ?Y ?Z ?M1 ?M2 ?A ?B)
8.   (DB ?M2) (GOAL (?X ?Y ?Z) (USE ?B))
9.   (DB ?M1) (NOT (GOAL (?X ?Y ?Z) (USE ?A))))
10. (LET S1 INFER (X Y) (COLDER ?X ?Y)
11.   (FACT (FURTHERNORTH ?X ?Y)))
12. (DB ?MT)
13. (ASSERT F1 F2 T1 T2)
14. (DB ?MS)
15. (ASSERT F1 F2 S1)
16. (DB ?MT)
17. (GOAL (CONTRADICTION ?X ?Y ?Z ?MT ?MS T1 S1) (USE T2))
18. (PRINT IS ?Y ?X THAN ?Z)
.
.
.
.
.
.

```

FIG. 2. The first question.

dialogue would suggest that the opening question is not a random one—it is apparently prompted by the belief that it will reveal to the student a "bug" in his computational procedure for deciding which of two places is likely to be colder. In particular, the teacher has hypothesized that the student "knows" that the further north of two places will be the colder, and has searched his data base for a contradiction to this theorem.

This process could be expressed in the present implementation as in Fig. 2. We are not concerned here with programming details but a brief

explanation is in order. The language may be considered to be a version of Micro-planner (Sussman *et al.*, 1972), except that the system maintains any number of "data bases" of knowledge, rather than just one to represent the system's knowledge. One therefore speaks in terms of solving a problem with respect to a particular data base. Here, we have two data bases: MT, which contains a representation of the teacher's (or teaching program's) knowledge, and MS, which contains a representation of the student's (hypothesized) knowledge. The latter is our "student model". As teaching proceeds, the contents of MT and MS are modified.

Knowledge is considered to be of two kinds: "facts", which are written as symbolic expressions without variables, and "theorems", which (at least, as far as Fig. 2 is concerned) are procedures for deducing facts from other facts. Facts and theorems may be named (lines 2-11) so that they may be asserted into data bases (lines 12-15)—unnamed facts and theorems may be asserted directly into only one data base. The theorem named T1 (line 4), for example, expresses the knowledge that "x is colder than y" may be inferred from the facts "x is further north than y" and "x is further inland than y". This theorem is asserted into MT (line 13). The above hypothesis concerning the student's knowledge state is expressed by asserting the theorem S1 into MS (line 15). Apart from domain-specific procedures such as T1, MT may also contain procedures representing type (a) knowledge. For example, T2 (line 6) is a procedure for deriving a "contradiction" between two procedures. It assigns values to the variables X, Y, Z* so that the fact (?X ?Y ?Z) may be proved with respect to (whatever data base has been assigned to) M2, using (whatever theorem has been assigned to) B, but not with respect to M1, using A. When called as on line 17, it assigns values such that the next line prints

IS NEWFOUNDLAND COLDER THAN MONTANA

The GOAL primitive initiates a search or modification of the current data base (using asserted theorems) to find an instantiation of its pattern argument. The further primitives INFER and FACT, respectively, inhibit the use of data base-modifying theorems and the use of any theorems at all. The system decides which theorems to apply by matching the calling (or "goal") pattern with the theorem templates [e.g. for T1 the template is (COLDER ?X ?Y)]. Theorems may be applied either to determine whether a particular fact is true or to assign values to unassigned variables so as to satisfy a calling

* The convention is that a variable is referenced by prefixing a ? and a literal atom is written explicitly. Thus (USE T2) on line 17 means "use the theorem named T2", and (USE ?A) on line 9 means "use the theorem whose name has been assigned to the variable A".

pattern. More complex "goals" may be specified by logical combinations of these primitives, and achieved by a system backtrack, if required, to find alternative instantiations for previous GOAL activations.

Regarding the implementation of the system, it is important of course that it should be possible to handle procedures as data objects, i.e. to create, modify and evaluate them. For example, to increase generality, the theorem S1 would not be explicitly defined as in Fig. 2; it would be created by a simple transformation of T1. Similarly, T1 would probably have been derived from a more complicated version. At each stage, the teaching program will use a procedure (e.g. T1) slightly more complicated than the student's analogue (e.g. S1). And, most important, as the student is deemed to learn, so the contents of MS are changed. For example, by line 24 of Fig. 1, the system should have deleted S1 from MS and asserted T1 into it. Thus MS is a dynamic model of the student, since at any instant it represents what the system thinks the student knows.

Feedback, Task Difficulty and Objectives

The conventional concerns of CAI can be re-expressed in terms of such a procedural student model.

The evidence suggests (e.g. Tait, Hartley & Anderson, 1973) that the effectiveness of an instructional program is largely determined by the program's ability to locate the causes of students' errors and to provide feedback which draws the students' active attention to these causes. An embodiment of these errors as procedural bugs gives scope for various kinds of feedback processes. Often, in CAI programs, feedback consists of a "trace", in which intermediate results are displayed to or asked of the student. This is, of course, always possible when type (b) knowledge is executed interpretively. However, such traces ought not, as they invariably are, to be independent of type (c) knowledge, i.e. the traces should, as far as possible, concentrate on a student's particular difficulties. The trace should, in other words, be a function of the difference between corresponding type (b) and type (c) procedures. Indeed, it may be more illuminating to a student to trace his own erroneous procedures.

Allied to such tracing feedback is "explanatory feedback", where the program attempts to explain, in natural language, the procedure it is attempting to teach. With explicit type (b) knowledge, we can avoid entirely pre-stored explanations (e.g. those of Peplinski), and attempt, as Siklossy proposes, "to explain to the student the problem-solving behaviour of the program when its behaviour can be fruitfully imitated by a human." Of

course this is one of the more intriguing problems in CAI, being intimately concerned with the design of high-level problem-oriented programming languages and with the mechanisms of human problem-solving. Any such generated explanations should again be functions of type (c) knowledge—when A explains something to B, A should bear in mind what B knows and what B does not know. The ability to generate procedural explanations is invaluable to a conversational CAI program; for example, in Fig. 1, sentence 5 may be construed as the teacher “explaining” procedure S1 to the student. Similarly, sentence 11 is an explanation of a simple transformation of S1. It is the function of type (a) knowledge to decide when to offer such explanations.

Similarly, task difficulty should not be defined only in terms of type (b) knowledge (as in, e.g. Peplinski, 1970), but also of type (c) knowledge (as in, e.g. Ayscough, 1973, where success probabilities are calculated from the values of registers serving as the student model). Here, we would express “difficulty” in terms of the complexity of the procedures in the student model which he is required to evaluate, and of the difference between corresponding teacher and student procedures.

A teaching program’s objectives, if they are expressed at all (and they are not by Carbonell, Wexler, or Brown), are usually specified as threshold register values. When, as here, we regard teaching as a process of modifying (a representation of) a student’s knowledge base, it is more natural to specify objectives in terms of the required contents of the student model, and hence to express type (a) knowledge in terms of “goals” to be achieved, i.e. in the same goal-oriented problem-solving formalism as for type (b) knowledge. Good teaching is a high-level intellectual activity requiring high-level languages for its adequate expression.

The teaching process above may be considered to be activated by a procedure call of the form (GOAL (?MS KNOWS T1)), where the body of the corresponding theorem describes the teaching strategy. If one now wants to impose more global goals, e.g. (GOAL (?MS THINKS SCIENTIFICALLY)), then the teaching programmer is forced to consider how he would specify procedure bodies to attain such goals. Sentences 20–32 of Fig. 1 are concerned with one particular procedure call within such a global goal, that is the conception and subsequent testing of hypotheses. This requires higher-order knowledge, i.e. knowledge about knowledge. Here, for example, the student (or MS) has been led to modify S1 temporarily to

```
(S2 INFER (X Y) (COLDER ?X ?Y)
  (FACT (EQUALNORTH ?X ?Y))
  (FACT (FURTHERINLAND ?X ?Y))),
```

i.e. to run a procedure expressing the knowledge that a hypothesis relating two factors may be tested by eliminating the effect of all other factors.

"Teaching goals" may therefore be expressed as non-deterministic procedures just like those used in problem-solving, and student errors may usefully be considered to initiate "failures", backtracking to remedial teaching.

A Program

A system has been implemented to provide a means of testing our ideas about teaching and student modelling, and a program written which attempts to teach along the lines discussed above. The printout shown in Fig. 3 shows the present ability of the program, which obviously, at the moment, serves only demonstration purposes. Of course, the surface appearance of an individual dialogue is important only in so far as it reflects the underlying methodology. Hence the attention is directed to the accompanying comments, which indicate how the program is constantly examining, modifying, evaluating and explaining a student model expressed as a set of procedures and facts. There are, at the moment, numerous trapdoors down which this "conversation" is liable to disappear, owing to the non-existence of any significant natural language analysis or teaching strategy component.

The system runs interactively on the Leeds Modular One multi-access system (Cole, 1973) and consists of a hierarchy of programming languages. At the highest level we have a PLANNER-like language (Hewitt, 1972), implemented as an extension of a slightly modified LISP 1.5 (McCarthy, Abrahams, Edwards, Hart & Levin, 1963) any LISP program may call a PLANNER function, and *vice versa*. Since PLANNER is an experimental language, we have not hesitated to modify functions or to add functions of our own. The LISP/PLANNER system runs under an interpreter written in BCPL (Richards, 1969). All functions which need to be explicitly available at run-time, to be modified, explained, etc. (e.g. the contents of the student model), are expressed in LISP/PLANNER, and any other functions (e.g. those expressing type (a) knowledge) may be written in LISP/PLANNER or in BCPL. The latter, being compiled, obviously run faster. There is no implied loss of power, since any LISP/PLANNER program may evaluate a BCPL function, and obviously *vice versa*. Owing to a hardware restriction limiting the size of a code segment to 8K, the system runs as a series of compiled segments, each of which may call other segments as subroutines. The consequent program overlaying deteriorates run times, and makes practical use of the system a dubious proposition.

IS NEWFOUNDLAND COLDER THAN MONTANA?

- ; question generated as described above, after
- ; deriving a contradiction between teacher model (MT)
- ; and hypothesized student model (MS)

YES←

WHY?

NEWFOUNDLAND IS FURTHER NORTH THAN MONTANA←

- ; linguistic analysis is currently rudimentary

YES. SO, IF YOU TAKE ANY TWO PLACES, IF THE FIRST PLACE IS FURTHER NORTH THAN THE SECOND PLACE THEN THE FIRST PLACE IS COLDER THAN THE SECOND PLACE?

- ; student's reply is checked, by examining MT.
- ; A LISP function generates explanations of
- ; procedures or LISP functions, under certain
- ; restrictions. Here, an explanation of S1
- ; has been embedded in the question. The
- ; quality of English generated is unsophisticated.

YES←

- ; confirms that MS should indeed contain S1

BUT MONTANA IS COLDER THAN NEWFOUNDLAND

WHAT DOES THAT IMPLY?

- ; system reports contradiction found above

I DON'T KNOW←

WOULD YOU SAY THAT IF THE FIRST PLACE IS FURTHER SOUTH THAN THE SECOND PLACE THEN THE FIRST PLACE IS COLDER THAN THE SECOND PLACE?

- ; system explains a transformed version of S1
- ; with NORTH replaced by its opposite SOUTH.

NO←

WOULD YOU SAY THAT BEING FURTHER NORTH HAS NO EFFECT?

- ; system knows that it has, since FURTHERNORTH
- ; occurs in T1. Type (a) knowledge should
- ; determine when to offer such hints.

NO←

WHAT OTHER FACTOR IS INVOLVED?

MONTANA IS FURTHER INLAND THAN NEWFOUNDLAND←

YES. IF THE FIRST PLACE IS FURTHER INLAND THAN THE SECOND PLACE THEN THE FIRST PLACE IS COLDER THAN THE SECOND PLACE

- ; another generated explanation—the system
- ; is at present linguistically incapable of
- ; accepting student sentences of such complexity.
- ; System has now modified MS, by modifying
- ; S1, to represent the student's new
- ; knowledge state.

.
.

.

.

.

FIG. 3. A Program-student dialogue.
(Student responses are terminated with a ←)

Some Criticisms

This section anticipates and comments upon some criticisms that may be made about the proposals of previous sections.

(a) The procedures in Fig. 2 are incorrect.

We are not concerned to provide a theory of meteorology, but only to indicate the advantages of manipulating explicit procedural models. However, to the extent that teaching requires a deep understanding of what is being taught, CAI programs will experience difficulties where explicit procedures cannot be specified.

(b) The program lacks a "natural language component".

Notwithstanding the claim of Taylor (1968) to have provided a "machine ability to 'follow' a line of argument along several or many alternative paths", there are considerable linguistic difficulties preventing the attainment of Fig. 1-type dialogues. Since effective communication demands that the speaker has a model of the knowledge of the listener (and *vice versa*), a procedural student model as here outlined seems a minimum prerequisite.

(c) The determination of program equivalence is a difficult problem.

The comparison of procedures as implied in previous sections does, indeed, amount to determining program equivalence which is a research area with barely significant results as yet. But, if this is what teaching involves—creating within a student procedures comparable to a teacher's—this difficulty must be acknowledged.

(d) The taxonomy of errors is incomplete.

In the discussion of Fig. 1, the only type of student error considered was "a conditional missing from a procedure". Clearly, there are other types of error, e.g. wrong function applied (e.g. confusion over "subset" and "member" in set theory), failure to generalize (i.e. procedure contains a constant which should be a variable), etc. One of the more interesting developments would be an attempt to define a taxonomy of errors in terms of a procedural model [cf. Carbonell's taxonomy, (1970, p. 90), which is clearly oriented toward "semantic network" analyses].

(e) There is no experimental evidence that a procedural model is a good descriptive one.

Since alternative models were criticized above on this account, this is a valid criticism not diminished by the fact that methods used by programs developed around procedural models (e.g. Winograd, 1972) are often compared to the corresponding problem-solving methods of

humans. Precise psychological evidence is lacking. Even so, this does not affect the principle that the student model should consist of a set of programs, but is concerned with details of the programs themselves.

Conclusions

The conclusions to be drawn from this study are threefold:

(a) A "student model" expressed in terms of a set of procedures in a high-level language is more insightful than most student models as implemented in existing CAI programs, and can lead to more sophisticated teaching performance, in that the operation of the teaching program can be made to depend upon an explicit representation of a student's current knowledge state.

(b) Certain approaches to CAI have been groping towards problems which are the essence of computer science itself, e.g. the equivalence of programs and the representation of problem-solving processes (or the design of programming languages), and there may be much to gain by acknowledging a common interest.

(c) There are considerable theoretical and practical difficulties to be overcome before CAI can achieve some of the goals that researchers have set for the field. These difficulties ensure that attempts, such as the above, to provide more pertinent question generation and feedback comment than is given by conventional CAI programs will, for some time, not be cost effective. There is, of course, little evidence that the intuition that more knowledgeable computer tutors do provide better instruction is a sound one. Until such programs are implemented there cannot be.

Acknowledgements

I am grateful to members of the Computer Based Learning Project, University of Leeds, for their comments on a draft of this paper, and also to Professor R. C. Anderson, University of Illinois, for permission to use the dialogue in Fig. 1.

References

- AYSCOUGH, P. B. (1973). Computer based learning in the teaching laboratory. *Chemistry in Britain*, **9**, 61.
- BROWN, J. S., BURTON, R. R. & ZDYBEL, F. (1972). A model-driven Question and Answering system for mixed initiative CAI. Technical Report No. 22, Dept. of Information and Computer Science, University of California, U.S.A.

- CARBONELL, J. R. (1970). Mixed initiative man-computer instructional dialogues. (PhD thesis, MIT) Report No. 1971. Cambridge, U.S.A.: Bolt Beranek & Newman.
- COLE, A. J. (1973). An outline description of the Leeds Modular One multi-access system. Internal report, Computer Based Learning Project, Leeds.
- COLES, L. S. (1969). Computer-aided instruction using an inferential question-answering system with natural language input: a plan for research. Technical note 11, AI Group, Stanford Res. Inst.
- COLLINS, A. M. & QUILLIAN, M. R. (1969). Retrieval time from semantic memory. *Journal of Verbal Learning and Verbal Behavior*, **8**, 240.
- HEWITT, C. (1972). Description and theoretical analysis (using schemata) of Planner. PhD thesis, MIT.
- MCCARTHY, J., ABRAHAMS, P. W., EDWARDS, D. J., HART, T. P. & LEVIN, M. I. (1963). LISP 1.5 Programmer's Manual. Cambridge, Mass.: MIT Press.
- NORMAN, D. A. (1972). Memory, knowledge and the answering of questions. Centre for Human Information Processing, Report 25, University of California.
- PEPLINSKI, C. (1970). A generative CAI program that teaches algebra. Technical Report No. 90, Computer Science Department, University of Wisconsin, Madison, U.S.A.
- QUILLIAN, M. R. (1969). The teachable language comprehender. *Comm. ACM*, **12**, 459.
- RICHARDS, M. (1969). BCPL Reference Manual. Technical Memorandum 69/1, Computer Laboratory, Cambridge.
- SIKLOSSY, L. (1971). Control and feedback in the environment of a computer tutor. *Proc. 4th Int. Symp. Amer. Soc. Cybernetics*.
- SUSSMAN, G. J., WINOGRAD, T. & CHARNIAK, E. (1972). Micro-planner Reference Manual. AI Memo 203A, MIT.
- SUSSMAN, G. J. & McDERMOTT, D. V. (1972). Conniver Reference Manual. AI Memo 259, MIT.
- TAIT, K., HARTLEY, J. R. & ANDERSON, R. C. (1973). Feedback procedures in computer-assisted arithmetic instruction. *British Journal of Educational Psychology*, **43**, 161.
- TAYLOR, E. F. (1968). Automated tutoring and its discontents. *American Journal of Physics*, **36**, 496.
- WEXLER, J. D. (1970). A generative, remedial and query system for teaching by computer. PhD thesis, University of Wisconsin, Madison, U.S.A.
- WINOGRAD, T. (1972). *Understanding natural language*. New York: Academic Press.